

Aperture — Database Design (Part 3)

Version: v1.0

Document Type: Database Design

31. Access Pattern Analysis

The database schema should be designed around how the application actually accesses data.

Rather than optimizing every possible query, the design prioritizes the most frequent and business-critical operations.

The dominant access patterns are:

User-centric

- Fetch profile by username
 - Retrieve user statistics
 - Load followers and following
 - Load user reviews
 - Load user lists
-

Content-centric

- Retrieve movie or TV details
 - Load cast and crew
 - Fetch reviews
 - Fetch streaming availability
 - Retrieve recommendations
-

Discovery-centric

- Homepage sections
 - Trending content
 - Editorial collections
 - Similar titles
-

Community-centric

- Like a review
- Follow a user
- Add to watchlist
- Create a custom list

The schema is optimized for these workflows first, while less common administrative or analytical queries are expected to use specialized reporting or background processing.

32. Query Design Philosophy

Queries should be:

- Predictable
- Index-friendly
- Pagination-first
- Explicit about relationships

Avoid:

- Deep nested joins when simpler alternatives exist
- N+1 query patterns
- Unbounded result sets
- Business logic embedded in SQL

Complex data composition belongs in the service layer, not in increasingly complex database queries.

33. Indexing Strategy

Indexes are introduced to support observed access patterns rather than hypothetical ones.

Core Indexes

Identity

- Email
- OAuth provider ID

Users

- Username
- Display name (optional for search)
- Profile visibility

Metadata

- TMDb ID
- Release date
- Popularity (if stored)
- Title (for lookups)

Reviews

- Movie ID + creation date
- User ID + creation date
- Rating

Lists

- Owner ID
- Visibility
- List item position

Streaming

- Movie/TV ID + Region
- Provider ID

Notifications

- Recipient ID + read status
- Creation date

Composite indexes should be added only when query profiling demonstrates repeated benefit.

34. Query Optimization

Optimization should follow a consistent progression:

1. Improve the query
2. Add an appropriate index
3. Cache the result
4. Denormalize if necessary
5. Partition or restructure only when justified

This order minimizes unnecessary complexity and keeps the schema maintainable.

35. Pagination Strategy

Every endpoint returning collections should support pagination.

Default approach:

- Cursor-based pagination for feeds, reviews, notifications, and activity streams
- Offset pagination for administrative interfaces where deterministic ordering is more important than performance

Cursor pagination provides more stable performance on large, frequently changing datasets.

36. Caching Strategy

Caching should accelerate expensive reads without becoming a second source of truth.

Good Cache Candidates

- Movie details
- TV details
- Trending sections
- Popular lists
- Public user profiles
- Streaming availability
- Search suggestions
- Discovery feeds

Avoid Caching

- Authentication state
- Highly personalized recommendations (unless precomputed)
- Active moderation queues
- Frequently edited reviews

All cached data should be disposable and reconstructable from authoritative sources.

37. Write Strategy

Writes should prioritize correctness over throughput.

Examples:

- Review creation

- Rating updates
- List modifications
- Follow relationships

Operations that trigger secondary effects (search indexing, recommendation updates, notifications) should persist the primary transaction first and then schedule follow-up work asynchronously where appropriate.

38. Read Strategy

Read operations should minimize latency through:

- Appropriate indexing
- Selective eager loading
- Lazy loading of secondary relationships
- Caching
- Parallel retrieval where multiple domains contribute to a response

The goal is to optimize the user-perceived experience rather than simply reducing the number of database queries.

39. Data Retention

Different categories of data require different retention policies.

Long-lived

- Movies
- TV Shows
- Users
- Reviews
- Lists

These form the core historical record of the platform.

Medium-lived

- Notifications
- Refresh sessions

These may be archived or purged after configurable periods.

Derived Data

- Search indexes
- Recommendation caches
- Embeddings

These can be regenerated if necessary.

Operational Data

- Audit logs
- Background job history

Retention should balance operational needs with storage costs.

40. Archival Strategy

Large datasets should be archived rather than deleted whenever possible.

Candidates include:

- Expired sessions
- Old notifications
- Historical audit logs
- Completed background job records

Archival should preserve historical integrity while reducing the size of frequently accessed tables.

41. Backup & Disaster Recovery

Production deployments should support:

- Automated daily backups
- Point-in-time recovery
- Periodic restore verification
- Backup retention policies

Backups are only considered reliable after successful restoration tests.

42. Scalability Roadmap

The persistence layer evolves incrementally.

Stage 1

Single PostgreSQL instance

Stage 2

Read replicas for heavy read traffic

Stage 3

Redis for caching

Stage 4

OpenSearch for advanced search

Stage 5

Partitioning of high-volume tables

Examples:

- Notifications
 - Audit logs
 - Activity events (future)
-

Stage 6

Dedicated vector storage (only if pgvector becomes insufficient)

The roadmap reflects measured growth rather than speculative optimization.

43. Observability

Database health should be continuously monitored.

Key metrics include:

- Query latency
- Slow query frequency
- Connection pool utilization
- Cache hit ratio
- Lock contention
- Replication lag (future)
- Storage growth

Monitoring should focus on trends rather than isolated spikes.

44. Operational Guidelines

Operational practices include:

- Review execution plans for slow queries
- Regular index maintenance
- Periodic schema review
- Controlled migration rollouts
- Capacity planning based on observed growth

Routine maintenance should be documented and automated where practical.

45. Database Evolution Principles

Schema evolution should be:

- Backward compatible where feasible
- Incremental
- Version controlled
- Reviewed
- Tested in staging

Avoid large, disruptive migrations whenever possible.

46. Interview Discussion Points

Be prepared to explain:

- Why optimize around access patterns instead of tables?
 - Why choose cursor pagination for community feeds?
 - When would you denormalize data?
 - Why separate operational and transactional data?
 - How would you scale PostgreSQL before introducing sharding?
 - How do you decide whether to cache a query?
-

47. Database Design Summary

Aperture's persistence layer is designed around domain ownership, relational integrity, and measured scalability.

The schema emphasizes correctness first, performance second, and complexity only when justified by real usage.

By aligning storage decisions with business domains, expected access patterns, and an incremental growth strategy, the database remains approachable for a single developer while providing a realistic path toward production-scale operation.

This concludes the Database Design document.